IQRASEC ACADEMY

Web Application Basics Report

WAPT Phase 1

Task 1 Web Fundamentals

Prepared by

Kinza Noor

Report date

3 October 2026

Learning platform

TryHackMe

Description:

This report explains the web communication concepts covered in the Web Application Basics room and documents its HTTP request exercises. It provides a foundation for interpreting web traffic during later web application penetration testing tasks.

Contents

1. Objective 3
Concept and Understanding 3
2.1 Web Application Architecture 3
2.2 URL Structure 3
2.3 HTTP and HTTPS 3
2.4 HTTP Message Structure 3
2.5 HTTP Methods 3
2.6 HTTP Status Codes 4
2.7 Headers Cookies and Sessions 4
2.8 Security Headers 4
3 Practical Work 4
3.1 Retrieving Users with GET 4
3.2 Updating a User with POST 4
3.3 Deleting a User with DELETE 5
3.4 Interpreting Requests and Responses 5
4 Tools Used 5
5 Evidence 5
5.1 GET Request Evidence 5
5.2 POST Request Evidence 5
5.3 DELETE Request Evidence 6
5.4 Room Completion Evidence 6
6 Result 6
7 Key Learnings 7
8 References 7

1. Objective

To understand how web applications operate, identify the components of URLs and HTTP messages, interpret request methods and response codes, and explain the purpose of common headers and browser security controls.

2. Concept and Understanding

2.1 Web Application Architecture

The front end is the part accessed through a browser. HTML provides page structure, CSS controls presentation, and JavaScript enables interaction. The back end processes requests and may retrieve or modify information in a database. A web server delivers content, while an optional Web Application Firewall filters potentially harmful requests.

2.2 URL Structure

https://example.com:443/profile?id=15#details

In this example, https is the scheme, example.com is the host, 443 is the port, /profile is the path, id=15 is the query parameter, and details is the fragment. The fragment is handled by the browser and is not sent as part of the HTTP request target.

2.3 HTTP and HTTPS

HTTP defines communication between clients and servers. A browser sends a request and receives a response. HTTPS protects communication using TLS encryption and server authentication. HTTPS protects transport; application access controls and input handling are still necessary.

2.4 HTTP Message Structure

An HTTP/1.1 message contains a start line, headers, a blank line and an optional body. A request line identifies the method, target and protocol version. A response status line identifies the version, status code and reason phrase. Bodies can contain HTML, JSON, form data or other content.

2.5 HTTP Methods

Method	Purpose
GET	Retrieve a representation of a resource.
POST	Submit data for processing, such as a form or API operation.
PUT	Create or replace the representation at a target resource.
PATCH	Apply partial changes to a resource.
DELETE	Request removal of a resource.
HEAD	Retrieve response metadata without a response body.
OPTIONS	Describe communication options for a resource.

2.6 HTTP Status Codes

Class	Meaning	Example
1xx	Informational response	100 Continue
2xx	Successful processing	200 OK
3xx	Redirection	301 Moved Permanently
4xx	Client error	404 Not Found
5xx	Server error	500 Internal Server Error

Status codes help explain how a request was handled. They should be considered alongside the response body and headers when checking application behavior.

2.7 Headers Cookies and Sessions

Request headers provide context: Host identifies the destination, User-Agent describes the client, and Content-Type describes the body format. Request bodies can use URL-encoded fields, multipart form data, JSON or XML. Response headers include Content-Type, Location and Set-Cookie.

The server uses Set-Cookie to request cookie storage; the browser returns applicable cookies in a Cookie header. A session identifier may link requests to server-side session state. Secure restricts cookie transmission to secure connections, while HttpOnly prevents JavaScript from reading the cookie.

2.8 Security Headers

Content-Security-Policy restricts allowed content sources. Strict-Transport-Security tells browsers to use HTTPS for an applicable host. X-Content-Type-Options with nosniff reduces MIME sniffing. Referrer-Policy controls referrer information shared with destinations. These protections complement secure application design.

3. Practical Work

Task 10 provides a browser-based emulator for making HTTP requests. The exercises cover reading data, updating a user and requesting deletion. The following descriptions identify each assigned action and the response evidence to record.

3.1 Retrieving Users with GET

GET /api/users

Objective: Retrieve the list of users.
Approach: Select GET, enter /api/users and send the request through the emulator.
Response review: Examine the returned user information and record the displayed result.
Evidence reference: Figure 1.

3.2 Updating a User with POST

POST /api/user/2
country=US

Objective: Change Bob's country from UK to US.
Approach: Select POST, enter /api/user/2, set country to US using the emulator controls and send the request.

Response review: Record the update response and any displayed confirmation.
Evidence reference: Figure 2.

The country value above describes the input required by the exercise; it is not a captured raw request. This emulator uses POST for the update, while other APIs may use PUT or PATCH.

3.3 Deleting a User with DELETE

DELETE /api/user/1

Objective: Request deletion of user 1.
Approach: Select DELETE, enter /api/user/1 and send the request.
Response review: Record the response indicating how the deletion request was handled.

3.4 Interpreting Requests and Responses

The method expresses the requested operation, the endpoint identifies the resource and the submitted data supplies operation-specific input. The response must be reviewed to verify the actual outcome. Successful processing should be demonstrated by the emulator result rather than inferred from the request alone.

4. Tools Used

Tool	Purpose
TryHackMe	Learning material and the Web Application Basics room.
Web browser	Access to the room and its interactive exercise.
HTTP request emulator	Selection of methods, entry of endpoints and inspection of results.

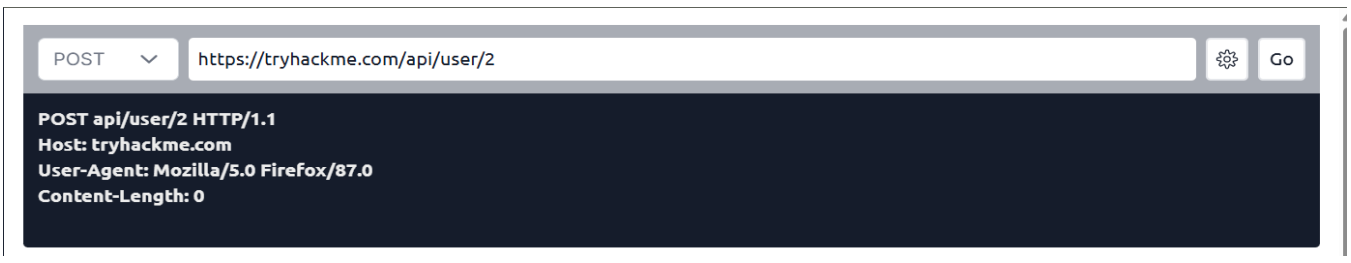
5. Evidence

Insert readable screenshots below, keeping each request and its corresponding result visible. The completion screenshot should accompany practical evidence.

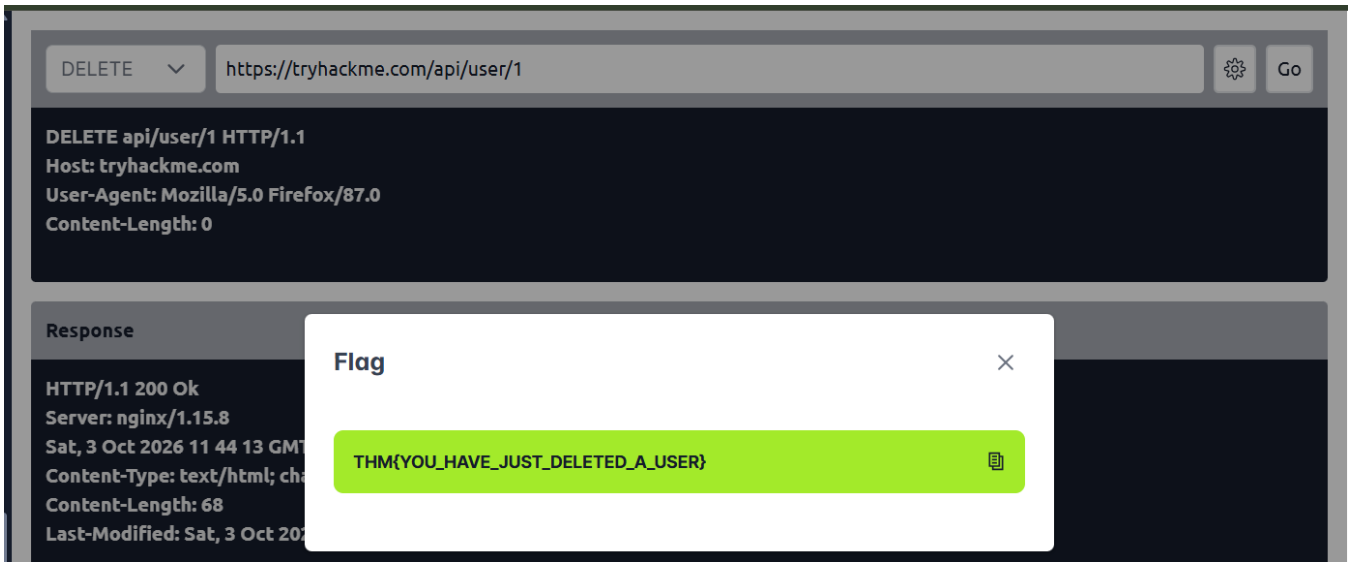
5.1 GET Request Evidence



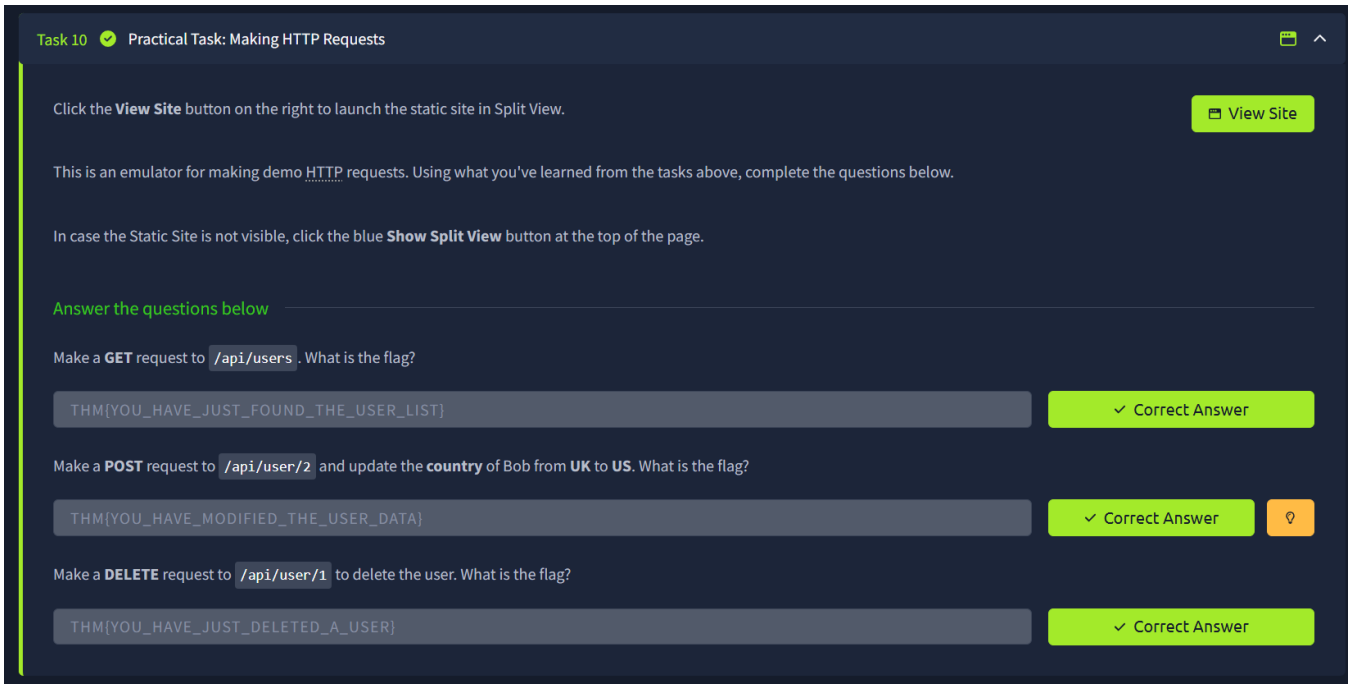
5.2 POST Request Evidence



5.3 DELETE Request Evidence



5.4 Room Completion Evidence



6. Result

I completed the TryHackMe Web Application Basics room. The exercises demonstrate how different HTTP methods operate on API resources, while the learning material explains the structure and security relevance of web communication. Add the supporting practical and completion evidence under Figures 1 to 4 before submission.

7. Key Learnings

Web requests must be interpreted through their methods, endpoints, headers and data. Responses reveal the result of processing and must be inspected before drawing conclusions. Cookies support continuity between requests, and HTTPS and security headers add protections. Server-side authorization remains necessary to restrict access to sensitive resources.

8. References

TryHackMe. Web Application Basics. https://tryhackme.com/room/webapplicationbasics

MDN Web Docs. Overview of HTTP. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview

MDN Web Docs. HTTP messages. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages

MDN Web Docs. HTTP reference. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference